

お土産 (Souvenirs)

Amaru が外国のお店でお土産を買っている。店内には N 種類のお土産があり、それぞれの種類のお土産が無数に用意されている。

お土産には種類に応じた値段が定められており、種類 i ($0 \leq i < N$) のお土産の値段は $P[i]$ である。ここで、 $P[i]$ は正の整数である。

Amaru は、お土産の種類は値段の高い順に並んでいること、そして各種類のお土産の値段は互いに異なることを知っている。すなわち、 $P[0] > P[1] > \dots > P[N-1] > 0$ である。さらに、彼は $P[0]$ の値を知ることにも成功した。しかし残念なことに、Amaru はお土産の値段に関する他の一切の情報を持っていない。

お土産を買うために、Amaru は売り手と何回かの取引を行う。

各取引は以下のステップで構成される：

1. Amaru が売り手に何枚かの（正の枚数の）コインを渡す。
2. 売り手がこれらのコインを、Amaru からは見えない奥の部屋のテーブルの上に山積みにする。
3. 売り手が、種類 $0, 1, \dots, N-1$ のお土産をこの順に一つずつ考慮する。各種類のお土産は各取引において **ちょうど一度だけ** 考慮される。
 - 種類 i のお土産を考慮するとき、もし現在山に積まれているコインの枚数が $P[i]$ 以上ならば、
 - 売り手が $P[i]$ 枚のコインを山から取り除き、
 - 売り手が種類 i のお土産を 1 つテーブルの上に置く。
4. 売り手が、山に積まれて残っているすべてのコインと、テーブルの上にあるすべてのお土産を Amaru に渡す。

なお、はじめテーブルの上にはコインもお土産も一切置かれていない。

あなたの課題は、以下の条件が共に満たされるよう、Amaru にいくつかの取引を行うよう指示することである。

- 各取引において、彼は **少なくとも 1 つ** のお土産を買う。
- $0 \leq i < N$ を満たす各 i について、彼は種類 i のお土産をすべての取引における合計で **ちょうど i 個** 買う。なおこれは、Amaru は種類 0 のお土産を 1 つも買ってはいけないことを意味する。

Amaru は取引回数を最小化する必要はなく、コインを無限に持っているものとする。

実装の詳細

あなたは以下の関数を実装する必要がある。

```
void buy_souvenirs(int N, long long P0)
```

- N : お土産の種類数.
- $P0$: $P[0]$ の値.
- この関数は各テストケースにおいてちょうど一度だけ呼び出される.

この関数は、Amaru に取引を指示するために以下の関数を呼び出すことができる。

```
std::pair<std::vector<int>, long long> transaction(long long M)
```

- M : Amaru から売り手に渡されるコインの枚数.
- この関数はペアを返す。ペアの最初の要素は、買われたお土産の種類を昇順に並べた配列 L である。ペアの二番目の要素は、取引後に Amaru に返されたコインの枚数を表す整数 R である.
- $P[0] > M \geq P[N - 1]$ が満たされていなければならない。 $P[0] > M$ という条件は Amaru が種類 0 のお土産を買わないことを保証し、 $M \geq P[N - 1]$ という条件は Amaru が少なくとも 1 つのお土産を買うことを保証する。これらの条件が満たされていない場合、あなたの解は `Output isn't correct: Invalid argument` の判定を受ける。なお、 $P[0]$ とは異なり、 $P[N - 1]$ の値は入力として与えられないことに注意せよ.
- この関数は各テストケースにおいて最大 5000 回まで呼び出すことができる.

採点プログラムは **適応的 (adaptive)** ではない。すなわち、価格の列 P は `buy_souvenirs` が呼び出される前から固定されている。

制約

- $2 \leq N \leq 100$.
- $1 \leq P[i] \leq 10^{15}$ ($0 \leq i < N$).
- $P[i] > P[i + 1]$ ($0 \leq i < N - 1$).

小課題

小課題	得点	追加の制約
1	4	$N = 2$.
2	3	$P[i] = N - i$ ($0 \leq i < N$).
3	14	$P[i] \leq P[i + 1] + 2$ ($0 \leq i < N - 1$).
4	18	$N = 3$.
5	28	$P[i + 1] + P[i + 2] \leq P[i]$ ($0 \leq i < N - 2$), $P[i] \leq 2 \cdot P[i + 1]$ ($0 \leq i < N - 1$).
6	33	追加の制約はない.

例

以下の呼び出しを考える.

```
buy_souvenirs(3, 4)
```

$N = 3$ 種類のお土産があり, $P[0] = 4$ である. このとき, 価格の列 P としてあり得るものは, $[4, 3, 2]$, $[4, 3, 1]$, $[4, 2, 1]$ の 3 通りである.

`buy_souvenirs` が `transaction(2)` を呼び出したとする. また, この呼び出しが $([2], 1)$ を返した, すなわち Amaru が種類 2 のお土産を 1 つ買い, 売り手が彼に 1 枚のコインを返したとする. このとき, 以下の理由により $P = [4, 3, 1]$ を導くことができる.

- $P = [4, 3, 2]$ の場合, `transaction(2)` は $([2], 0)$ を返したはずである.
- $P = [4, 2, 1]$ の場合, `transaction(2)` は $([1], 0)$ を返したはずである.

その後, `buy_souvenirs` は `transaction(3)` を呼び出すことができる. この呼び出しは $([1], 0)$ を返し, Amaru が種類 1 のお土産を 1 つ買い, 売り手が彼に 0 枚のコインを返したことを意味する. この時点までの合計で, 彼は種類 1 と種類 2 のお土産を 1 つずつ買ったことになる.

最後に, `buy_souvenirs` は `transaction(1)` を呼び出すことができる. この呼び出しは $([2], 0)$ を返し, Amaru が種類 2 のお土産を 1 つ買ったことを意味する. なお, この呼び出しの代わりに `transaction(2)` を呼び出すことも可能である. この時点までの合計で, 彼は種類 1 のお土産を 1 つと種類 2 のお土産を 2 つ買ったことになり, これは問題の要求に合致する.

採点プログラムのサンプル

入力形式:

N

$P[0] \ P[1] \ \dots \ P[N-1]$

出力形式：

$Q[0] \ Q[1] \ \dots \ Q[N-1]$

ここで、 $0 \leq i < N$ を満たす各 i について、 $Q[i]$ は種類 i のお土産が合計で買われた回数を表す。